

CROSSLEDGER

Cross-Border Multi-Currency Wallet & Remittance Platform

Complete Technology Stack & Business Logic Specification

Full-Stack FinTech Engineering Portfolio Project

Ameer Maavia
.NET Developer | FinTech & SaaS

Islamabad, Pakistan
[linkedin.com/in/ameer-maavia](https://www.linkedin.com/in/ameer-maavia)

1. Project Overview

CrossLedger is a production-grade cross-border payments platform that allows users to hold multiple currency wallets, obtain live foreign exchange quotes from external rate providers, and transfer money between currencies through a double-entry accounting ledger. It is built as a full-stack solution: an ASP.NET Core Web API backend, a Blazor WebAssembly frontend, a SQL Server database, and a complete Azure cloud deployment with automated CI/CD.

Unlike typical portfolio applications, CrossLedger is not a CRUD demonstration. Its value lies in solving the genuinely difficult engineering problems that real financial systems face: exchange rate volatility, duplicate transaction requests, concurrent balance modification, provider outages, and auditable money movement. Every one of these problems has a deliberate, defensible solution in the codebase.

Design Principle — Every design decision in this project exists to answer a specific interview question. The goal is not feature count, but demonstrable depth in financial systems engineering.

2. Core Business Logic

This is the section that differentiates the project. Each rule below models how real payment infrastructure behaves.

2.1 Double-Entry Ledger

Wallet balances are never stored as a mutable column. Instead, every movement of money writes immutable ledger entries, and a balance is derived as the sum of its entries. A single cross-currency transfer produces four entries, which together must always sum to zero.

LEDGER ENTRIES — single transfer

Transfer: 100 USD -> PKR @ rate 278.50

Entry 1	DEBIT	User USD Wallet	-100.00 USD
Entry 2	CREDIT	FX Settlement (USD)	+100.00 USD
Entry 3	DEBIT	FX Settlement (PKR)	-27,850.00 PKR
Entry 4	CREDIT	User PKR Wallet	+27,850.00 PKR

Invariant: $\text{SUM}(\text{entries}) \text{ per currency} = 0$ (always)

Why this matters: a stored balance can drift, be overwritten, or lose its history. A derived balance is auditable, reconstructable, and mathematically verifiable. An automated test asserts the zero-sum invariant across the entire database after every test run.

2.2 FX Quote Locking & Spread

Exchange rates move continuously. A user must not be able to view a rate, wait, and then execute against a stale favourable rate. CrossLedger issues a quote with a fixed expiry window; the transfer endpoint accepts a quote identifier, not a raw amount, and rejects expired quotes.

```
QUOTE_LIFECYCLE
POST /api/v1/quotes { from: USD, to: PKR, amount: 100 }

mid-market rate from provider -> 279.90
platform spread (0.50%) -> 278.50 (revenue margin)
quoteId c9f2... expiresAt +30 seconds

POST /api/v1/transfers { quoteId: c9f2..., idempotencyKey: ... }
-> if utcNow > expiresAt -> 409 QUOTE_EXPIRED
```

The spread is how real remittance providers earn revenue: the customer receives a rate slightly worse than the mid-market rate, and the difference is recorded as platform income in its own ledger account. This makes the project a business model, not just an API.

2.3 Idempotency

Networks fail after a request is processed but before the response arrives. The client retries. Without protection, the customer is debited twice. Every mutating endpoint requires an Idempotency-Key header, persisted with a unique database constraint. A repeated key returns the original stored response rather than executing again.

2.4 Concurrency Control

Two transfers initiated simultaneously from the same wallet must not both succeed against the same balance. CrossLedger uses EF Core optimistic concurrency with a RowVersion token on the wallet aggregate. On conflict, the operation is retried with exponential backoff; if the balance is genuinely insufficient on retry, it fails cleanly with a domain error rather than corrupting state.

2.5 Provider Resilience

The platform integrates two independent exchange-rate providers behind a single abstraction. If the primary provider is slow or unavailable, Polly opens a circuit breaker and traffic is routed to the secondary provider. If both are unreachable, the most recent cached rate is served with an explicit staleness flag, and transfers above a configured threshold are blocked.

```
RESILIENCE_CHAIN
IExchangeRateProvider
|
|-- ExchangeRateApiProvider (primary - 160+ currencies)
|-- FrankfurterProvider (fallback - ECB, no API key)

Policy: Retry(3, exponential) -> CircuitBreaker -> Fallback
Cache: IMemoryCache, TTL 1 hour, stale-if-error
```

2.6 Additional Domain Rules

- **Currency precision** — money is never stored as float; a Money value object enforces decimal precision and currency-specific scale (JPY has 0 decimals, USD has 2, KWD has 3).

- **Cross-currency arithmetic is illegal** — the type system prevents adding USD to PKR; an explicit conversion through a quote is required.
- **Transfer limits** — configurable per-transaction and rolling 24-hour limits per user, modelling basic AML controls.
- **Immutable history** — ledger entries are append-only; corrections are made by posting a reversing entry, never by updating or deleting.
- **Webhook delivery** — HMAC-signed payloads with retry and exponential backoff, mirroring how Stripe and similar providers notify merchants.

3. Complete Technology Stack

Every technology below appears in the codebase and can be defended in an interview.

3.1 Frontend

Technology	Purpose	Why it is used in CrossLedger
Blazor WebAssembly	Single-page application framework in C#	Full-stack in one language; no context switching; runs client-side in the browser
Razor Components	Reusable UI component model	Wallet cards, quote widget, transaction table built as isolated components
MudBlazor	Material Design component library	Professional UI without writing CSS from scratch; data grids, dialogs, charts
Blazored.LocalStorage	Browser storage abstraction	Persist JWT access token across page refreshes
ApexCharts / MudBlazor Charts	Data visualisation	FX rate history line chart, balance breakdown, transaction volume
SignalR Client	Real-time messaging	Live rate ticker and instant transaction status updates without polling
FluentValidation (shared)	Client + server validation	Same validation rules compiled to both sides — no duplication

3.2 Backend & API

Technology	Purpose	Why it is used in CrossLedger
.NET 8 / C# 12	Runtime and language	Latest LTS; records, pattern matching, required members used throughout
ASP.NET Core Web API	HTTP API framework	RESTful endpoints with versioning (/api/v1), content negotiation
MediatR	In-process CQRS mediator	Commands (CreateTransfer) separated from Queries (GetBalance)
FluentValidation	Request validation	Validation as pipeline behaviour, not scattered in controllers
AutoMapper	Object mapping	Domain entities mapped to DTOs without manual assignment code
Polly	Resilience policies	Retry, circuit breaker and fallback for external FX provider calls
Hangfire	Background job scheduler	Webhook retries, rate refresh, pending transfer settlement
Serilog	Structured logging	Correlation IDs across requests; JSON sinks to Application Insights

Technology	Purpose	Why it is used in CrossLedger
Swashbuckle / OpenAPI	API documentation	Interactive Swagger UI with JWT auth support
Refit	Typed HTTP client	Declarative interfaces for external FX and payment provider calls

3.3 Data & Persistence

Technology	Purpose	Why it is used in CrossLedger
Microsoft SQL Server	Primary relational database	ACID transactions are non-negotiable for ledger integrity
Entity Framework Core 8	Object-relational mapper	Migrations, change tracking, optimistic concurrency via RowVersion
Dapper	Micro-ORM for reads	High-performance raw SQL for balance aggregation and reporting queries
Redis (StackExchange)	Distributed cache	FX rate caching and idempotency key storage across instances
EF Core Global Query Filters	Multi-tenancy enforcement	Tenant isolation applied automatically at the data layer

3.4 Security & Authentication

Technology	Purpose	Why it is used in CrossLedger
JWT Bearer Tokens	Stateless authentication	Access token with short expiry plus refresh token rotation
ASP.NET Core Identity	User and role management	Password hashing, logout, email confirmation
Role-Based Access Control	Authorisation	Customer, Support and Admin roles with policy-based endpoints
Azure Key Vault	Secret management	Connection strings and API keys never stored in appsettings.json
HMAC-SHA256	Webhook signing	Receivers verify payload authenticity, as Stripe requires
Rate Limiting Middleware	Abuse prevention	Built-in .NET 8 fixed and sliding window limiters per endpoint
Data Protection API	Encryption at rest	Sensitive fields encrypted with keys managed in Key Vault

3.5 External Integrations

Service	Role	Rationale
ExchangeRate-API	Primary FX rate provider	1,500 free requests per month, 160+ currencies, any base currency
Frankfurter API	Fallback FX provider	Open source, unlimited, no API key; ECB-sourced daily rates
Stripe (test mode)	Card funding simulation	Wallet top-up flow with webhook confirmation handling
SendGrid	Transactional email	Transfer receipts and security notifications

3.6 Testing

Technology	Purpose	Why it is used in CrossLedger
xUnit	Test framework	Facts and Theories across domain, application and API layers
Moq	Mocking library	Isolate services from repositories and external providers
FluentAssertions	Readable assertions	Expressive failure messages that document intent
WebApplicationFactory	In-memory API host	Full HTTP integration tests without deploying
Testcontainers	Real SQL Server in Docker	Integration tests run against actual SQL Server, not a fake provider
Bogus	Test data generation	Realistic fake users, wallets and transactions for seeding
Coverlet + ReportGenerator	Coverage measurement	Coverage gate enforced in the CI pipeline
NBomber	Load testing	Concurrent transfer simulation to prove the concurrency control works

3.7 Cloud & Infrastructure (Microsoft Azure)

Service	Purpose	Why it is used in CrossLedger
Azure App Service	API hosting	Managed hosting with deployment slots for zero-downtime releases
Azure Static Web Apps	Blazor WASM hosting	Global CDN distribution of the compiled front end
Azure SQL Database	Managed database	Automated backups, point-in-time restore, geo-redundancy
Azure Cache for Redis	Managed cache	Shared cache across scaled-out API instances
Azure Key Vault	Secrets and certificates	Accessed via Managed Identity — no credentials in code
Azure Application Insights	APM and telemetry	Distributed tracing, dependency maps, custom metrics, live alerts
Azure Functions	Serverless jobs	Scheduled FX rate refresh and nightly ledger reconciliation
Azure Service Bus	Message broker	Reliable event publishing for transfer completed / failed events
Azure Blob Storage	Object storage	Generated PDF statements and exported transaction reports

3.8 DevOps & Tooling

Technology	Purpose	Why it is used in CrossLedger
Docker	Containerisation	Multi-stage Dockerfile; identical image from local to production
Docker Compose	Local orchestration	API, SQL Server and Redis started with a single command
GitHub Actions	CI/CD pipeline	Build, test, coverage gate, container publish, Azure deploy
Azure DevOps Pipelines	Alternative CI/CD	Documented as a second pipeline definition for enterprise parity
Bicep	Infrastructure as Code	Entire Azure environment reproducible from a single template

Technology	Purpose	Why it is used in CrossLedger
Git / GitHub	Version control	Conventional commits, protected main branch, pull request reviews
SonarCloud	Static code analysis	Code smells, duplication and maintainability tracked per commit

4. Data Access Strategy — EF Core vs Stored Procedures

CrossLedger deliberately uses two data access approaches side by side. This is not indecision — it reflects how production financial systems are actually built. The decision rule is simple: if the work is about a single entity and its business rules, EF Core handles it. If the work is set-based, spans millions of rows, or must execute as one atomic multi-statement unit, it belongs in a stored procedure.

Approach	When it is used	Examples in CrossLedger
Entity Framework Core	Single entity or small set. Business rules applied in the domain. Change tracking and migrations matter.	Creating a wallet, registering a user, updating a quote, saving a webhook subscription
Dapper + Stored Procedure	Set-based aggregation, complex joins, window functions, or multiple statements that must be atomic.	Posting a transfer, reconciling the ledger, generating statements, paged transaction history

Decision rule — If the data never needs to reach C# — if the answer is a computed result rather than a set of objects — it should be computed in SQL. Pulling two million ledger rows into memory to sum them is the mistake this project is designed to avoid.

4.1 Stored Procedures in CrossLedger

Stored Procedure	Responsibility	Why a stored procedure rather than EF Core
usp_PostTransfer	Atomic money movement	Validates balance, acquires row locks, inserts four ledger entries and the idempotency record inside a single server-side transaction. One round trip instead of five.
usp_GetWalletBalance	Balance derivation	Sums ledger entries forward from the latest snapshot. Returns a single scalar rather than transferring the entire entry history to the application.
usp_GetTransactionHistory	Server-side paging	Filtered, sorted, joined and paged in SQL with OFFSET/FETCH. Avoids the poor execution plans EF Core produces for deep pagination with multiple joins.
usp_ReconcileLedger	Nightly integrity check	Aggregates every ledger entry grouped by currency and asserts the sum is zero. Pure set-based work over the full table — impossible to do efficiently in C#.
usp_GetAccountStatement	Monthly statement	Produces a running balance using SUM() OVER (ORDER BY PostedAt). Window functions cannot be expressed cleanly through LINQ.
usp_CheckTransferLimits	AML limit enforcement	Computes a rolling 24-hour transfer total via an indexed seek before a transfer is authorised. Executes in microseconds.
usp_GetFxRateOHLC	Rate chart data	Aggregates raw rate snapshots into daily open, high, low and close values for the front-end chart using window functions.

4.2 Reference Implementation

The transfer procedure demonstrates the patterns expected in production SQL: explicit transaction handling, error trapping, row locking, and a deterministic error contract returned to the caller.

```
usp_PostTransfer.sql
CREATE PROCEDURE dbo.usp_PostTransfer
    @TransferId      UNIQUEIDENTIFIER,
    @FromWalletId    UNIQUEIDENTIFIER,
    @ToWalletId      UNIQUEIDENTIFIER,
    @SourceAmount     DECIMAL(19,4),
    @TargetAmount     DECIMAL(19,4),
    @Rate             DECIMAL(18,8),
    @IdempotencyKey   VARCHAR(64)
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- lock the source wallet for the duration of the transaction
        DECLARE @Balance DECIMAL(19,4);
        SELECT @Balance = Balance
        FROM dbo.WalletBalances WITH (UPDLOCK, ROWLOCK)
        WHERE WalletId = @FromWalletId;

        IF @Balance < @SourceAmount
        BEGIN
            ROLLBACK TRANSACTION;
            THROW 50001, 'INSUFFICIENT_FUNDS', 1;
        END

        -- four entries: debit source, credit FX, debit FX, credit target
        INSERT INTO dbo.LedgerEntries
            (EntryId, TransferId, WalletId, Direction, Amount, CurrencyCode, PostedAt)
        VALUES
            (NEWID(), @TransferId, @FromWalletId, 'DEBIT', @SourceAmount, ...),
            (NEWID(), @TransferId, @FxSettlementId, 'CREDIT', @SourceAmount, ...),
            (NEWID(), @TransferId, @FxSettlementId, 'DEBIT', @TargetAmount, ...),
            (NEWID(), @TransferId, @ToWalletId, 'CREDIT', @TargetAmount, ...);

        INSERT INTO dbo.IdempotencyKeys (KeyValue, TransferId, CreatedAt)
        VALUES (@IdempotencyKey, @TransferId, SYSUTCDATETIME());

        COMMIT TRANSACTION;
        SELECT @TransferId AS TransferId, 'COMPLETED' AS Status;
    END TRY
    BEGIN CATCH
        IF XACT_STATE() <> 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END
```

4.3 Calling Procedures from the Application

Stored procedures are invoked through Dapper behind a repository interface, so the application layer remains unaware of the data access mechanism and stays fully testable.

```

SqlTransferRepository.cs – Dapper
public async Task<TransferResult> PostTransferAsync(PostTransferCommand cmd, CT ct)
{
    using var conn = new SqlConnection(_connectionString);

    var parameters = new DynamicParameters();
    parameters.Add("@TransferId", cmd.TransferId);
    parameters.Add("@FromWalletId", cmd.FromWalletId);
    parameters.Add("@SourceAmount", cmd.SourceAmount, DbType.Decimal, precision: 19, scale: 4);
    parameters.Add("@IdempotencyKey", cmd.IdempotencyKey);

    return await conn.QuerySingleAsync<TransferResult>(
        "dbo.usp_PostTransfer",
        parameters,
        CommandType.StoredProcedure);
}

```

4.4 SQL Engineering Practices Demonstrated

- **Explicit transactions** — BEGIN TRY / CATCH with XACT_ABORT and XACT_STATE checks so a failure never leaves a partial transaction open.
- **Locking hints** — UPDLOCK and ROWLOCK on the balance read prevent two concurrent transfers from both passing the balance check.
- **SET NOCOUNT ON** — suppresses row-count messages that otherwise add network chatter on high-frequency procedures.
- **Deterministic error contract** — THROW with defined error numbers mapped to domain exceptions in C#, rather than parsing message strings.
- **Indexing strategy** — covering index on (WalletId, PostedAt) INCLUDE (Amount, Direction) so balance and statement queries are index-only scans.
- **Execution plan analysis** — plans captured before and after indexing, with the improvement documented in the repository.
- **Parameter sniffing awareness** — OPTION (RECOMPILE) applied selectively on reporting procedures where parameter values vary widely.
- **Migrations include procedures** — stored procedures are versioned as SQL scripts executed by EF Core migrations, so the database is reproducible from source control.

5. Payment Provider Orchestration

A single payment provider cannot serve every corridor, currency and settlement speed. Real cross-border platforms integrate several providers behind one abstraction and select between them at runtime. This orchestration layer is the most architecturally significant part of CrossLedger and the clearest demonstration of senior-level design.

Principle — The provider name on an integration is not what makes a project senior. The routing engine that decides which provider to call, what to do when it fails, and how to prove the decision afterwards — that is what makes it senior.

5.1 Provider Selection

All providers listed below offer publicly available developer sandboxes, so every integration in this project is genuinely executable rather than mocked.

Provider	Role	Capability and sandbox availability
Airwallex	Primary — global payouts & FX	Multi-currency accounts, international transfers with conversion quotes and live rates. Sandbox at api.sandbox.airwallex.com . Idempotency via <code>request_id</code> ; webhooks signed HMAC-SHA256 with an <code>x-signature</code> header.
Rapyd	Secondary — alternate corridors	Sandbox environment that generates webhooks exactly as production does, allowing full failover testing.
Stripe	Funding rail	Card top-up into wallets, with signature-verified webhook confirmation. Well-documented test mode.
SimulatedProvider	Chaos and test harness	In-process provider that deliberately injects timeouts, partial failures, delayed settlement and duplicate webhooks to prove the system handles them.

5.2 The Abstraction

Every provider implements one interface. Adding a fifth provider requires no change to any existing class — a direct, demonstrable application of the Open/Closed Principle.

```
IPaymentProvider.cs
public interface IPaymentProvider
{
    ProviderCode Code { get; }

    // can this provider serve this corridor at all?
    bool Supports(Corridor corridor);

    // what would it cost and how long would it take?
    Task<ProviderQuote> QuoteAsync(PayoutRequest req, CT ct);

    // execute — must be idempotent on req.IdempotencyKey
    Task<PayoutResult> ExecuteAsync(PayoutRequest req, CT ct);

    // verify an inbound webhook actually came from this provider
    bool VerifySignature(string payload, string signature, string secret);

    Task<ProviderHealth> CheckHealthAsync(CT ct);
}
```

5.3 Routing Engine

The routing engine evaluates every eligible provider against a weighted strategy and records why the winner was chosen. Every routing decision is persisted, so any transfer can be explained months later — which is an audit requirement in regulated payments.

Signal	Role in scoring	How it is evaluated
Corridor support	Hard filter	Provider is excluded outright if it cannot serve the source/target currency pair and destination country
Health status	Hard filter	Excluded while its circuit breaker is open or its last health check failed
Total cost	Weighted 40%	Fixed fee plus FX spread, normalised into the source currency for fair comparison
Settlement speed	Weighted 30%	Estimated arrival time; weighting increases when the user selects a priority transfer
Historical reliability	Weighted 20%	Rolling success rate over the last 24 hours, computed from stored payout outcomes

Signal	Role in scoring	How it is evaluated
Limit headroom	Weighted 10%	Remaining daily volume allowance with that provider

```

PaymentRoutingEngine.cs
public async Task<RoutingDecision> SelectAsync(PayoutRequest req, CT ct)
{
    var eligible = _providers
        .Where(p => p.Supports(req.Corridor))
        .Where(p => _health.IsAvailable(p.Code))
        .ToList();

    if (eligible.Count == 0)
        throw new NoRouteAvailableException(req.Corridor);

    var quotes = await Task.WhenAll(
        eligible.Select(p => p.QuoteAsync(req, ct)));

    var scored = quotes
        .Select(q => new { Quote = q, Score = _scorer.Score(q, req.Preference) })
        .OrderByDescending(x => x.Score)
        .ToList();

    // persist the full comparison, not just the winner
    await _auditLog.RecordRoutingAsync(req.TransferId, scored, ct);

    return new RoutingDecision(
        Primary: scored[0].Quote,
        Fallback: scored.Skip(1).Select(x => x.Quote).ToList());
}

```

5.4 Failure Handling

Routing produces an ordered chain, not a single choice. If the primary provider fails, execution moves down the chain — but only when it is provably safe to retry.

- **Deterministic failure** (rejected, unsupported, validation error) — fail over immediately to the next provider in the chain.
- **Timeout or ambiguous response** — do *not* fail over blindly. The payout may have succeeded. The system first queries the provider using the original idempotency key to establish the true state, and only then decides.
- **All providers exhausted** — the transfer is marked PENDING_MANUAL, funds stay reserved but unspent, and an operations alert is raised. Money is never silently lost.
- **Duplicate webhooks** — events are deduplicated on provider event id before any ledger entry is written, because providers do not guarantee once-only delivery.
- **Out-of-order webhooks** — a settled event arriving before a processing event must not regress the transfer state; state transitions are validated against an explicit state machine.

5.5 Transfer State Machine

TRANSFER LIFECYCLE

QUOTED

| accept (quote still valid)

v

RESERVED funds held, ledger entries written

| route selected

v

SUBMITTED -----> PROVIDER_FAILED ---> retry next provider in chain

|

|

+-----> PENDING_MANUAL (chain exhausted)

v

PROCESSING provider acknowledged, awaiting settlement

v

SETTLED terminal – webhook confirmed

Any state -> REVERSED posts compensating ledger entries;
original entries are never deleted

Transitions are enforced in the domain layer, not by convention. An invalid transition throws a domain exception, and a unit test exists for every illegal path.

5.6 Chaos Testing

The SimulatedProvider exists so that failure paths are tested rather than assumed. It is configured per test to reproduce the failure modes that real providers exhibit.

Scenario	What is simulated	What must hold true
Timeout after acceptance	Provider accepts then never responds	System must query by idempotency key rather than blindly retry
Duplicate webhook	Same event delivered three times	Ledger must be written exactly once
Out-of-order events	settled arrives before processing	State machine must reject the regression
Partial provider outage	Primary fails, secondary healthy	Routing must fail over and complete the transfer
Total outage	Every provider unavailable	Transfer parks at PENDING_MANUAL with funds reserved
Concurrent transfers	Fifty simultaneous payouts, one wallet	NBomber proves no overdraft and no ledger imbalance

6. Authentication & Step-Up Security

Authentication in a payments platform is not a single gate at login. Sensitive operations are re-challenged individually, because a session that was valid an hour ago is not sufficient justification for revealing a card number or moving a large sum. CrossLedger implements step-up authentication, which is standard practice in regulated financial products and rarely seen in portfolio projects.

6.1 Authentication Factors

Factor	Implementation	Behaviour
Password	ASP.NET Core Identity	Argon2-class hashing, lockout after repeated failures, breach-list rejection on registration
TOTP	Otp.NET	RFC 6238 time-based codes, compatible with Google Authenticator and Authy; QR provisioning on enrolment

Factor	Implementation	Behaviour
Recovery codes	Own implementation	Ten single-use codes issued at enrolment, stored hashed, consumed atomically
Trusted device	Signed device token	Optional thirty-day remember-this-device token, revocable individually from the security page
Step-up token	Short-lived JWT	Issued only after a fresh TOTP verification; valid five minutes; scoped to a single operation class

6.2 Step-Up Policy

Which operations require re-authentication is configuration, not scattered conditionals. The policy below is data-driven and can be adjusted per tenant without a code change.

Operation	Step-up	Reasoning
View balances and history	No	Read-only, no financial or data-exposure risk
Transfer below threshold	No	Low value; friction would outweigh the risk reduction
Transfer above threshold	Yes	Primary loss vector if a session is hijacked
Add or edit a beneficiary	Yes	Classic attack path — redirect future transfers to an attacker account
Reveal card PAN or CVC	Yes	Direct exposure of payment credentials
Change spending limits	Yes	Raising a limit is a precursor to a larger unauthorised spend
Disable two-factor auth	Yes	Protects the protection itself
Change password or email	Yes	Account takeover prevention

6.3 Declarative Enforcement

Step-up is enforced by an authorisation filter driven by an attribute, so a reviewer can see an endpoint's security requirement without reading its body.

```
TransfersController.cs
[HttpPost("transfers")]
[Authorize(Roles = "Customer")]
[RequireStepUp(Operation = StepUpOperation.HighValueTransfer,
    ThresholdSetting = "Limits:StepUpAbove")]
public async Task<IActionResult> Create(CreateTransferRequest req, CT ct)
{
    // the filter has already validated:
    // - a valid access token
    // - the Customer role
    // - a step-up token, fresh within 5 minutes, scoped to this operation
    var result = await _mediator.Send(req.ToCommand(), ct);
    return Ok(result);
}
```

If the step-up token is missing or stale the request is rejected with HTTP 403 and a machine-readable code, allowing the Blazor client to open the verification dialog and retry without losing the user's input.

6.4 Security Controls

- **Token pair** — short-lived access token with a rotating refresh token; reuse of a consumed refresh token revokes the entire family, which detects theft.

- **TOTP replay protection** — a consumed code is recorded and cannot be presented twice within its validity window.
- **Rate limiting on verification** — a small number of TOTP attempts before temporary lockout, preventing brute force across the code space.
- **Enrolment integrity** — the TOTP secret is only persisted after the user proves they can generate a valid code, so a mis-scanned QR never locks anyone out.
- **Full audit trail** — every challenge, success, failure, device trust grant and revocation is recorded with IP, user agent and timestamp.
- **Secrets isolation** — TOTP secrets encrypted at rest with keys held in Azure Key Vault, never in the application database in plaintext.

7. Solution Architecture

The solution follows Clean Architecture. Dependencies point inward only: the Domain layer references nothing, and the outermost layers depend on abstractions defined further in.

SOLUTION STRUCTURE

CrossLedger.sln

```
src/
  CrossLedger.Domain      <- entities, value objects, domain events
                           NO external dependencies at all

  CrossLedger.Application <- CQRS handlers, interfaces, validators
                           depends on: Domain

  CrossLedger.Infrastructure <- EF Core, Redis, Dapper, email
                              depends on: Application, Domain

  CrossLedger.Providers   <- Airwallex, Rapyd, Stripe, Simulated
                           each behind IPaymentProvider

  CrossLedger.Api         <- controllers, middleware, DI wiring
                           depends on: Application, Infrastructure

  CrossLedger.Client      <- Blazor WebAssembly front end
  CrossLedger.Shared      <- DTOs and validators used by API + Client

tests/
  CrossLedger.Domain.Tests <- pure unit tests, no mocks needed
  CrossLedger.Application.Tests <- unit tests with Moq
  CrossLedger.Integration.Tests <- WebApplicationFactory + Testcontainers

infra/
  main.bicep              <- all Azure resources as code
  docker-compose.yml
```

7.1 Request Flow — Executing a Transfer

```

END-TO-END FLOW
Blazor Client
  | POST /api/v1/transfers + Idempotency-Key + Bearer token
  v
API Controller      -> JWT validation, rate limiting, model binding
  v
MediatR Pipeline    -> logging -> validation -> idempotency check
  v
CreateTransferHandler -> load quote, assert not expired
                    -> load wallets with RowVersion
                    -> Domain: Wallet.Debit() / Wallet.Credit()
                    -> build 4 ledger entries, assert zero-sum
  v
EF Core Transaction -> persist entries + idempotency record atomically
  v
Domain Event        -> TransferCompleted -> Service Bus
  v
Hangfire Job         -> signed webhook delivery with retry
  v
SignalR              -> push live status back to the Blazor client

```

8. CV Skills Mapping

Each claim on the CV is evidenced by a specific, reviewable part of this repository.

CV Skill	Where it lives	What it demonstrates
C# / .NET 8	Entire solution	Records, pattern matching, async streams, minimal hosting
ASP.NET Core Web API	CrossLedger.Api	Versioned REST endpoints, custom middleware, problem details
Blazor WebAssembly	CrossLedger.Client	Full SPA: auth, dashboard, live rates, transaction history
Entity Framework Core	Infrastructure/Persistence	Migrations, RowVersion concurrency, global query filters
SQL Server	Database project	Indexed ledger tables, covering indexes, execution plan tuning
Stored Procedures	db/procedures/*.sql	Seven procedures: atomic transfer posting, reconciliation, statements, limits
T-SQL	usp_PostTransfer, usp_GetAccountStatement	Transactions, locking hints, THROW error contract, SUM() OVER window functions
Dapper	SqlTransferRepository	Typed parameter binding and procedure invocation behind a repository interface
Clean Architecture	Solution layout	Four projects with enforced inward dependency direction
SOLID Principles	IXchangeRateProvider	New provider added without modifying existing code
CQRS	Application layer	Commands and Queries handled separately via MediatR
Unit Testing	Domain + Application tests	60+ tests covering money arithmetic and transfer rules
Integration Testing	Integration.Tests	Real SQL Server via Testcontainers, full HTTP round trips
Microservices concepts	Service Bus events	Event-driven boundaries, idempotent consumers
Docker	Dockerfile, compose	Multi-stage build, non-root user, health checks
CI/CD	GitHub Actions	Test gate, coverage threshold, automated Azure deployment
Microsoft Azure	infra/main.bicep	Nine services provisioned entirely as infrastructure as code

CV Skill	Where it lives	What it demonstrates
JWT / OAuth 2.0	Auth module	Access and refresh token rotation with reuse detection
Two-factor authentication	Security module	TOTP enrolment, recovery codes, trusted devices, replay protection
Step-up authentication	RequireStepUp filter	Operation-scoped re-authentication driven by declarative policy
Card issuing (Phase 2)	CrossLedger.Cards	Stripe Issuing sandbox: virtual and physical lifecycle, spending controls
Real-time authorisation	Issuing webhook handler	Sub-two-second approve or decline against live derived balance
RBAC	Authorisation policies	Customer, Support and Admin separation enforced per endpoint
Multi-tenancy	Tenant middleware	Resolved per request, isolated by global query filter
Payment APIs	CrossLedger.Providers	Airwallex and Rapyd sandbox integrations plus Stripe funding rail
Payment orchestration	PaymentRoutingEngine	Weighted multi-signal provider selection with audited routing decisions
Resilience engineering	SimulatedProvider, Polly	Chaos tests for timeouts, duplicate webhooks and total provider outage
State machines	Transfer aggregate	Enforced lifecycle transitions with a unit test per illegal path
Webhooks	Outbound delivery	HMAC signing, retry with exponential backoff, dead letter

9. Front-End Scope

The Blazor client is deliberately small but complete — enough to prove full-stack capability without consuming the timeline.

Screen	Type	What it demonstrates
Login / Register	Auth	JWT handling, refresh token rotation, protected route guards
Wallet Dashboard	Core	Multi-currency balance cards, total value in a chosen base currency
Send Money	Core	Two-step quote-then-confirm flow with a live 30-second countdown timer
Transaction History	Core	Server-side paged data grid with filtering and CSV export
Ledger Viewer	Differentiator	Expandable double-entry breakdown per transaction — shows accounting depth
FX Rates	Core	Live rate ticker over SignalR plus 30-day historical chart
Security Settings	Auth	TOTP enrolment with QR, recovery codes, trusted device list with revocation
Step-Up Dialog	Auth	Modal TOTP challenge that preserves form state and retries the original request
Cards (Phase 2)	Phase 2	Card list, PAN reveal behind step-up, freeze toggle, spending control editor
Routing Inspector	Differentiator	Per-transfer view of every provider quote considered and why the winner was selected
Admin Console	RBAC proof	User management, transfer limits, provider health, webhook delivery log

10. Delivery Roadmap

Week	Milestone	Deliverable
1–2	Foundation	Solution structure, Domain layer, Money and Currency value objects, unit tests for money arithmetic
3	Ledger engine	Ledger entities, double-entry posting logic, zero-sum invariant test
3b	SQL layer	usp_PostTransfer, usp_GetWalletBalance, covering indexes, execution plan comparison
4	FX layer	Two providers behind one interface, Polly resilience chain, caching, quote locking
5	Application layer	MediatR commands and queries, validation pipeline, idempotency behaviour
6	Provider layer	IPaymentProvider abstraction, Airwallex and Rapyd sandbox clients, webhook verification
6b	Routing engine	Corridor matrix, weighted scoring, fallback chain, routing audit log
7	API layer	Controllers, JWT auth, RBAC policies, rate limiting, Swagger
7b	Security	TOTP enrolment, recovery codes, step-up token issuance, RequireStepUp filter
8	Testing	Integration tests with Testcontainers, chaos scenarios, NBomber concurrency proof
9–10	Blazor front end	Auth, dashboard, send-money flow, ledger viewer, charts, SignalR
11	Cloud deploy	Bicep templates, Key Vault, App Insights, Azure Functions, Static Web Apps
12	CI/CD	GitHub Actions pipeline, deployment slots, SonarCloud analysis
13	Presentation	README with architecture diagrams, API docs, demo video, LinkedIn series
14–20	Phase 2	Card issuing module — detailed separately in section 12

11. Interview Defence

The purpose of the project is to make these answers concrete rather than theoretical.

Likely question	Answer grounded in the codebase
How do you prevent a duplicate charge on retry?	Idempotency-Key header persisted with a unique constraint; a repeated key replays the stored response instead of re-executing.
Two transfers hit the same wallet simultaneously — what happens?	EF Core RowVersion optimistic concurrency. The loser gets a concurrency exception and is retried with backoff; insufficient funds then fails cleanly.
Where is a wallet balance stored?	It is not stored. It is derived by summing immutable ledger entries, with periodic snapshots for read performance.
Your FX provider goes down mid-transaction. Then what?	Polly retries, opens a circuit breaker, and fails over to a second provider. If both are down, a cached rate is served with a staleness flag and large transfers are blocked.
How do you test a service without touching a database?	The handler depends on interfaces only. Moq supplies the repository; domain rules are tested with no infrastructure at all.
Where do your secrets live?	Azure Key Vault, accessed through Managed Identity. No connection string or API key appears in appsettings.json or in source control.
How do you know the ledger is never out of balance?	An integration test asserts that the sum of all ledger entries per currency equals zero after every scenario, and a database check constraint enforces it.
Why Clean Architecture and not a single project?	Domain rules must be testable without a database or HTTP context. Inward-only dependencies make that possible and keep infrastructure replaceable.

Likely question	Answer grounded in the codebase
When do you use a stored procedure instead of EF Core?	When the work is set-based or must be atomic across statements. Transfer posting and nightly reconciliation are procedures; wallet creation and quote storage are EF Core.
Why not put all business logic in stored procedures?	Domain rules in SQL cannot be unit tested without a database and are invisible to the type system. Procedures handle data mechanics; the domain layer owns the rules.
Why re-authenticate a user who is already logged in?	A session proves who authenticated an hour ago, not who is acting now. High-value transfers, beneficiary changes and PAN reveal each require a fresh TOTP, scoped to that operation for five minutes.
A virtual card has no PIN. How is that handled in code?	Capabilities are declared per card type in one table and evaluated centrally, so no conditional branches are duplicated across the API and the UI.
A card is presented and you have two seconds. What happens?	The webhook handler makes no external calls. It verifies the signature, checks card status and spending controls, reads balance through an index built for that path, then approves with a fund reservation.
How do you choose which payment provider to use?	A routing engine filters by corridor support and health, then scores the remainder on cost, speed, reliability and limit headroom. The full comparison is persisted for audit.
Your provider times out. Do you retry with another one?	Not immediately. A timeout is ambiguous — the payout may have succeeded. The system first queries the provider with the original idempotency key to establish the true state.
What happens if every provider is down?	The transfer parks at PENDING_MANUAL with funds reserved but unspent, and an alert is raised. Money is never silently lost or double-spent.
A webhook arrives twice. What breaks?	Nothing. Events are deduplicated on provider event id before any ledger entry is written, because providers do not guarantee once-only delivery.
How do you stop two concurrent transfers overdrawing a wallet?	The balance is read with UPDLOCK and ROWLOCK inside the procedure transaction, so the second transfer waits and then fails the balance check cleanly.

12. Phase Two — Card Issuing

Card issuing is deliberately scheduled as a second phase rather than bundled into the initial build. The core platform — ledger, FX, orchestration and security — is delivered and deployed first; cards are then added on top of it. This sequencing is itself a talking point: it demonstrates incremental delivery against a stable foundation, and the repository history shows a real system evolving rather than appearing fully formed.

12.1 Platform

Stripe Issuing is used because sandbox access is granted instantly from the dashboard without a sales process. In the sandbox every card transaction is simulated and no real funds move, and both virtual and physical cards can be created and exercised through their full lifecycle.

Scope note — Physical cards in the sandbox progress through their real lifecycle states but are never manufactured or shipped. The repository documents this explicitly — the implementation is complete, the fulfilment is simulated.

12.2 Capability Model

Virtual and physical cards do not support the same operations. A virtual card has no PIN because it is never presented at a terminal. Rather than encoding this as conditional logic scattered through controllers and UI, capabilities are declared per card type in a single configuration table and evaluated centrally. Adding a card type requires no change to any existing branch.

Capability	Virtual	Physical	Notes
View details (PAN / CVC)	Yes	Yes — sandbox	Live mode restricts unredacted retrieval to virtual cards
Freeze / unfreeze	Yes	Yes	Immediate authorisation block, reversible by the cardholder
Spending controls	Yes	Yes	Per-transaction, daily and monthly caps; merchant category and country rules
Transaction history	Yes	Yes	Derived from authorisation and capture events, linked to ledger entries
Change PIN	No	Yes	A virtual card is never presented at a terminal, so it has no PIN
Activation	Automatic	On delivery	Physical cards remain inactive until the cardholder confirms receipt
Replacement	Reissue	Ship new	Lost or stolen physical cards are cancelled and a replacement is ordered

12.3 Real-Time Authorisation

The most technically significant part of the card module is the authorisation webhook. When a card is presented, the issuer asks the platform in real time whether to approve, and the response must be returned within a strict time budget. This is where the card module and the ledger meet.

```

CARD AUTHORISATION FLOW
  card presented at merchant
    |
    v
  issuer -> POST /webhooks/issuing/authorization (budget: ~2 seconds)
    |
    +-- verify webhook signature
    +-- resolve card -> wallet
    +-- check card status (frozen? expired? not activated?)
    +-- check spending controls (per-txn, daily, monthly, MCC, country)
    +-- check available balance (derived, not cached stale)
    |
    v
  APPROVE -> reserve funds, write pending ledger entries
  DECLINE -> return decline reason code, write audit record only
    |
    v
  later: capture webhook -> convert pending entries to settled
        or expiry        -> release the reservation

```

- **Time budget** — the handler performs no external calls; balance is read through an indexed query designed for this path specifically.
- **Reservation, not deduction** — an approved authorisation holds funds. Settlement may arrive days later, may differ in amount, or may never arrive at all.
- **Partial and incremental captures** — a fuel pump authorises one amount and captures another; the ledger must reconcile the difference rather than assume equality.
- **Expiry sweeper** — a scheduled job releases reservations whose authorisation window has lapsed without capture, returning the funds to available balance.
- **Decline reasons are recorded** — every decline stores the specific rule that rejected it, so support can answer why a card was refused.

12.4 Phase Two Scope

Week	Milestone	Deliverable
14–15	Card domain	Card aggregate, capability model, status state machine, spending control rules with unit tests
16	Stripe Issuing	Cardholder creation, virtual card issuance, PAN reveal behind step-up authentication
17	Authorisation	Real-time authorisation webhook, fund reservation, decline reason codes
18	Settlement	Capture and expiry handling, partial capture reconciliation, reservation sweeper job
19	Physical cards	Order, fulfilment state tracking, delivery activation, PIN management, replacement flow
20	Front end	Card list, reveal dialog, freeze toggle, spending control editor, card transaction view

Summary — CrossLedger is not built to be a large application. It is built to be a deep one — a repository where every file can be explained, every decision justified, and every CV skill demonstrated in running code.